

Веб-застосунок для відстеження харчування та калорій, побудований на **ASP.NET Core 9** за архітектурою **Clean Architecture**.

Архітектура

Проект складається з чотирьох шарів:

```
StayFit.Domain      <- Сутності, інтерфейси репозиторіїв (без залежностей)
StayFit.Application  <- Сервіси, бізнес-логіка, DTOs
StayFit.Infrastructure <- EF Core, PostgreSQL, реалізації репозиторіїв
StayFit.Web          <- ASP.NET Core MVC, DI-контейнер, точка входу
```

Залежності між шарами

```
Web -> Infrastructure -> Application -> Domain
Web -> Application -> Domain
```

Ключові компоненти

Шар	Що містить
Domain	<code>User</code> , <code>Food</code> , <code>FoodLog</code> (сутності); <code>IUserRepository</code> , <code>IFoodRepository</code> , <code>IFoodLogRepository</code> (інтерфейси)
Application	<code>LoggingService</code> — сервіс з структурованим логуванням
Infrastructure	<code>AppDbContext</code> , <code>Repository<T></code> (базовий), <code>UserRepository</code> , <code>FoodRepository</code> , <code>FoodLogRepository</code>
Web	<code>Program.cs</code> — реєстрація всіх сервісів у DI; Serilog; MVC-контролери

Технологічний стек

- **Runtime:** .NET 9
- **Web-фреймворк:** ASP.NET Core MVC
- **ORM:** Entity Framework Core 9 + Npgsql (PostgreSQL)
- **Логування:** Serilog (Console + File + Seq)
- **БД:** PostgreSQL

Essentials для запуску

- .NET 9 SDK
 - PostgreSQL (версія 14+)
 - *(Опційно) Seq для централізованого перегляду логів
-

Запуск проекту

1. Клонування / розпакування

```
git clone <repo-url>  
cd StayFit
```

2. Налаштування бази даних (через secrets, без коміту пароля)

Важливо: рядок підключення не зберігаємо в `appsettings.json`.

Перейдіть у папку веб-проекту:

```
cd src/StayFit/StayFit.Web
```

Запишіть connection string у локальні user-secrets:

```
dotnet user-secrets set "ConnectionStrings:DefaultConnection" "Host=ep-ancient-  
tooth-an0wah2y-pooler.c-6.us-east-  
1.aws.neon.tech;Database=neondb;Username=neondb_owner;Password=  
<YOUR_PASSWORD>;Ssl Mode=Require;Channel Binding=Require;"
```

Перевірка:

```
dotnet user-secrets list
```

3. Застосування міграцій

```
cd StayFit.Web  
dotnet ef database update --project ../StayFit.Infrastructure
```

4. Запуск у потрібному середовищі

PowerShell:

```
$env:ASPNETCORE_ENVIRONMENT="Development"
dotnet run --project src/StayFit/StayFit.Web
```

Git Bash (MINGW64):

```
export ASPNETCORE_ENVIRONMENT=Development
dotnet run --project src/StayFit/StayFit.Web
```

Для Staging/Production просто замініть значення змінної:

- Development
- Staging
- Production

Щоб не залежати від `launchSettings.json`, можна запускати так:

```
ASPNETCORE_ENVIRONMENT=Staging dotnet run --project src/StayFit/StayFit.Web --no-launch-profile --urls "http://localhost:5250"
```

Після запуску відкривайте адресу з логів `Now listening on: ....`

Примітка: `App:BaseUrl` в `appsettings.*.json` це конфіг для посилань/інтеграцій, а не адреса, на якій сам сервер починає слухати порт.

Реєстрація сервісів у DI

Усі залежності реєструються у `Program.cs` через extension-метод `AddInfrastructure()`:

```
// Infrastructure (БД+ репозиторії)
builder.Services.AddInfrastructure(builder.Configuration);

// Application сервіси
builder.Services.AddScoped<LoggingService>();
```

Метод `AddInfrastructure (StayFit.Infrastructure/DependencyInjection.cs)` реєструє:

```
services.AddDbContext<AppDbContext>(...);
services.AddScoped(typeof(IRepository<>), typeof(Repository<>));
services.AddScoped<IUserRepository, UserRepository>();
services.AddScoped<IFoodRepository, FoodRepository>();
services.AddScoped<IFoodLogRepository, FoodLogRepository>();
```

Логування

Використовується **Serilog** з трьома sink-ами:

Sink	Опис
Console	Виводить у термінал у форматі [HH:mm:ss LVL] повідомлення
File	Записує у <code>logs/stayfit-YYYYMMDD.txt</code> (rolling by day)
Seq	Надсилає структуровані логи на <code>http://localhost:5341</code>

Приклад використання `LoggingService` у контролері:

```
public class HomeController : Controller
{
    private readonly LoggingService _loggingService;

    public HomeController(LoggingService loggingService)
    {
        _loggingService = loggingService;
    }

    public async Task<IActionResult> Index()
    {
        var result = await _loggingService.ProcessDataAsync("test");
        return View();
    }
}
```

Структура бази даних

```
Users          (Id, Name, Email, CreatedAt)
Foods          (Id, Name, CaloriesPer100g, ProteinPer100g, FatPer100g,
CarbsPer100g)
FoodLogs       (Id, UserId → Users, FoodId → Foods, AmountGrams, LoggedAt)
```

Міграції знаходяться у `StayFit.Infrastructure/Migrations/`.

Структура репозиторіїв

```
IRepository<T>      <- GetByIdAsync, GetAllAsync, AddAsync, UpdateAsync,
DeleteAsync
└─ IUserRepository <- + GetByEmailAsync, GetUsersWithFoodLogsAsync
```

```
|— IFoodRepository    <- + SearchByNameAsync  
|— IFoodLogRepository <- + GetByUserIdAsync, GetByUserIdAndDateAsync
```

Опціональний запуск Seq (для перегляду логів)

```
docker run -d \  
  --name seq \  
  -e ACCEPT_EULA=Y \  
  -p 5341:80 \  
  datalust/seq:latest
```

Після запуску Seq доступний за адресою: <http://localhost:5341>